

Experiment ‘p_5_5_repeat’ Results

June 6, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 1

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 3

Hard fails (resolution): 0

Soft fails (resolution): 1

Verification attempts: 2

Problem Specification

Problem name: p_5_5_repeat

Natural language statement: Write a method that returns a string repeated n times. For example, repeat("ho", 3) returns "hohoho".

Method signature: p_5_5_repeat(str: string, n: int) returns (repeated_str: string)

Ensures

- `|repeated_str| == n * |str|`
- forall `i :: 0 <= i < n ==> repeated_str[i * |str| .. (i + 1) * |str| ] == str`

Requires

- `n >= 0`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that returns a string repeated n times. For example, `repeat("ho", 3)`
↪ returns "hohoho".

The signature should be:

`p_5_5_repeat(str: string, n: int) returns (repeated_str: string)`

The method should respect the following contract:

requires $n \geq 0$, ensures $|repeated_str| == n * |str|$, ensures forall $i :: 0 \leq i < n \Rightarrow$
↪ $repeated_str[i * |str| .. (i + 1) * |str|] == str$

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnjL6UmIc1oBS1YFbEidkpZWvjrkJ

Model: gpt-5.5-2026-04-23

Created at: 1780744728

Finish reason: stop

Usage: 378 tokens in, and 4016 tokens out

```
{
  repeated_str := "";
  var k := 0;

  while k < n
    invariant 0 <= k <= n
    invariant |repeated_str| == k * |str|
    invariant forall j :: 0 <= j < k ==> repeated_str[j * |str| .. (j + 1) * |str|] == str
  {
    var old_repeated := repeated_str;
    assert |old_repeated| == k * |str|;
    assert forall j :: 0 <= j < k ==> old_repeated[j * |str| .. (j + 1) * |str|] == str;

    repeated_str := old_repeated + str;
    assert repeated_str == old_repeated + str;
    assert |repeated_str| == |old_repeated| + |str|;
    assert |repeated_str| == (k + 1) * |str|;

    forall j: int | 0 <= j < k + 1
```

```

    ensures repeated_str[j * |str| .. (j + 1) * |str|] == str
  {
    if j < k {
      var lo := j * |str|;
      var hi := (j + 1) * |str|;
      assert 0 <= lo <= hi;
      assert hi <= k * |str|;
      assert hi <= |old_repeated|;
      assert repeated_str[lo .. hi] == old_repeated[lo .. hi];
      assert old_repeated[lo .. hi] == str;
    } else {
      assert j == k;
      var lo := j * |str|;
      var hi := (j + 1) * |str|;
      assert lo == |old_repeated|;
      assert hi == |old_repeated| + |str|;
      assert hi == |repeated_str|;
      assert repeated_str[lo .. hi] == str;
    }
  }

  k := k + 1;
}
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```
problem_data/p_5_5_repeat/program.dfy(24,26): Error: lower bound out of range
```

```

|
24 | ensures repeated_str[j * |str| .. (j + 1) * |str|] == str
| ^

```

```
problem_data/p_5_5_repeat/program.dfy(30,8): Error: assertion might not hold
```

```

|
30 | assert hi <= k * |str|;
| ~~~~~

```

Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
 → Dafny body, including the curly braces that surround it. Do not show the signature
 → nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnjOLfR0iwUXU7hzcS00Ry0jFkXbG

Model: gpt-5.5-2026-04-23

Created at: 1780744929

Finish reason: stop

Usage: 1055 tokens in, and 7304 tokens out

```
{
  var L := |str|;
  repeated_str := "";
  var k := 0;

  while k < n
    invariant L == |str|
    invariant 0 <= L
    invariant 0 <= k <= n
    invariant |repeated_str| == k * L
    invariant forall j :: 0 <= j < k ==> (0 <= j * L <= (j + 1) * L <= |repeated_str| &&
      ↪ repeated_str[j * L .. (j + 1) * L] == str)
  {
    var old_repeated := repeated_str;
    assert |old_repeated| == k * L;
    assert forall j :: 0 <= j < k ==> (0 <= j * L <= (j + 1) * L <= |old_repeated| &&
      ↪ old_repeated[j * L .. (j + 1) * L] == str);

    repeated_str := old_repeated + str;
    assert repeated_str == old_repeated + str;
    assert |repeated_str| == |old_repeated| + L;
    assert (k + 1) * L == k * L + L;
    assert |repeated_str| == (k + 1) * L;

    forall j: int
      ensures 0 <= j < k + 1 ==> (0 <= j * L <= (j + 1) * L <= |repeated_str| &&
        ↪ repeated_str[j * L .. (j + 1) * L] == str)
    {
      if 0 <= j < k + 1 {
        assert j <= k;
        assert 0 <= j * L;
        assert (j + 1) * L == j * L + L;
        assert j * L <= (j + 1) * L;
        assert 0 <= k - j;
        assert k + 1 == (j + 1) + (k - j);
        assert (k + 1) * L == (j + 1) * L + (k - j) * L;
        assert 0 <= (k - j) * L;
        assert (j + 1) * L <= (k + 1) * L;
        assert 0 <= j * L <= (j + 1) * L <= |repeated_str|;

        if j < k {
          var lo := j * L;
          var hi := (j + 1) * L;
          assert j + 1 <= k;
          assert 0 <= k - (j + 1);
          assert k == (j + 1) + (k - (j + 1));
          assert k * L == (j + 1) * L + (k - (j + 1)) * L;
```

```

    assert 0 <= (k - (j + 1)) * L;
    assert hi <= k * L;
    assert hi <= |old_repeated|;
    assert repeated_str[.. |old_repeated|] == old_repeated;
    assert repeated_str[lo .. hi] == repeated_str[.. |old_repeated|][lo .. hi];
    assert repeated_str[.. |old_repeated|][lo .. hi] == old_repeated[lo .. hi];
    assert old_repeated[lo .. hi] == str;
    assert repeated_str[lo .. hi] == str;
  } else {
    assert j == k;
    var lo := j * L;
    var hi := (j + 1) * L;
    assert lo == |old_repeated|;
    assert hi == |old_repeated| + L;
    assert hi == |repeated_str|;
    assert repeated_str[|old_repeated| .. |repeated_str|] == str;
    assert repeated_str[lo .. hi] == str;
  }
}

k := k + 1;
}

assert k == n;
assert L == |str|;
assert |repeated_str| == n * |str|;
assert forall j :: 0 <= j < n ==> repeated_str[j * |str| .. (j + 1) * |str|] == str;
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_5_5_repeat/program.dfy(15,14): Warning: Could not find a trigger for this
  ↳ quantifier. Without a trigger, the quantifier may cause brittle verification. To
  ↳ silence this warning, add an explicit trigger using the {:trigger} attribute. For
  ↳ more information, see the section on quantifier instantiation rules in the
  ↳ reference manual.
|
15 | invariant forall j :: 0 <= j < k ==> (0 <= j * L <= (j + 1) * L <= |repeated_str| &&
  ↳ repeated_str[j * L .. (j + 1) * L] == str)
| ~~~~~
problem_data/p_5_5_repeat/program.dfy(19,11): Warning: Could not find a trigger for this
  ↳ quantifier. Without a trigger, the quantifier may cause brittle verification. To
  ↳ silence this warning, add an explicit trigger using the {:trigger} attribute. For
  ↳ more information, see the section on quantifier instantiation rules in the
  ↳ reference manual.
|

```

```

19 | assert forall j :: 0 <= j < k ==> (0 <= j * L <= (j + 1) * L <= |old_repeated| &&
    |   ↪ old_repeated[j * L .. (j + 1) * L] == str);
    |   ~~~~~~

problem_data/p_5_5_repeat/program.dfy(27,4): Warning: Could not find a trigger for this
    |   ↪ quantifier. Without a trigger, the quantifier may cause brittle verification. To
    |   ↪ silence this warning, add an explicit trigger using the {:trigger} attribute. For
    |   ↪ more information, see the section on quantifier instantiation rules in the
    |   ↪ reference manual.
    |
27 | forall j: int
    |   ~~~~~~

```

Compilation failed because warnings were found and --allow-warnings is false

Correct the error in code by altering only the method body. Produce and show only the

- ↪ Dafny body, including the curly braces that surround it. Do not show the signature
- ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnjQzLefQ23tU1Nxe9a4SLbnB2I8X

Model: gpt-5.5-2026-04-23

Created at: 1780745093

Finish reason: stop

Usage: 2741 tokens in, and 9044 tokens out

```

{
  var L := |str|;
  repeated_str := "";
  var k := 0;

  while k < n
    invariant L == |str|
    invariant 0 <= L
    invariant 0 <= k <= n
    invariant |repeated_str| == k * L
    invariant forall j {:trigger repeated_str[j * L .. (j + 1) * L]} :: 0 <= j < k ==> (0 <=
      |   ↪ j * L <= (j + 1) * L <= |repeated_str| && repeated_str[j * L .. (j + 1) * L]
      |   ↪ == str)
    {
      var old_repeated := repeated_str;
      assert |old_repeated| == k * L;
      assert forall j {:trigger old_repeated[j * L .. (j + 1) * L]} :: 0 <= j < k ==> (0 <=
        |   ↪ j * L <= (j + 1) * L <= |old_repeated| && old_repeated[j * L .. (j + 1) * L] ==
        |   ↪ str);

      repeated_str := old_repeated + str;
      assert repeated_str == old_repeated + str;
      assert |repeated_str| == |old_repeated| + L;
    }
}

```

```

assert (k + 1) * L == k * L + L;
assert |repeated_str| == (k + 1) * L;

forall j: int {:trigger repeated_str[j * L .. (j + 1) * L]} | 0 <= j < k + 1
  ensures 0 <= j * L <= (j + 1) * L <= |repeated_str| && repeated_str[j * L .. (j + 1)
    ↪ * L] == str
{
  assert j <= k;
  assert 0 <= j * L;
  assert (j + 1) * L == j * L + L;
  assert j * L <= (j + 1) * L;
  assert 0 <= k - j;
  assert k + 1 == (j + 1) + (k - j);
  assert (k + 1) * L == (j + 1) * L + (k - j) * L;
  assert 0 <= (k - j) * L;
  assert (j + 1) * L <= (k + 1) * L;
  assert 0 <= j * L <= (j + 1) * L <= |repeated_str|;

  if j < k {
    var lo := j * L;
    var hi := (j + 1) * L;
    assert j + 1 <= k;
    assert 0 <= k - (j + 1);
    assert k == (j + 1) + (k - (j + 1));
    assert k * L == (j + 1) * L + (k - (j + 1)) * L;
    assert 0 <= (k - (j + 1)) * L;
    assert hi <= k * L;
    assert hi <= |old_repeated|;
    assert old_repeated[lo .. hi] == str;
    assert repeated_str[.. |old_repeated|] == old_repeated;
    assert repeated_str[lo .. hi] == repeated_str[.. |old_repeated|][lo .. hi];
    assert repeated_str[.. |old_repeated|][lo .. hi] == old_repeated[lo .. hi];
    assert repeated_str[lo .. hi] == str;
  } else {
    assert j == k;
    var lo := j * L;
    var hi := (j + 1) * L;
    assert lo == |old_repeated|;
    assert hi == |old_repeated| + L;
    assert hi == |repeated_str|;
    assert repeated_str[|old_repeated| .. |repeated_str|] == str;
    assert repeated_str[lo .. hi] == str;
  }
}

k := k + 1;
}

assert k == n;
assert L == |str|;
assert |repeated_str| == n * |str|;

```

```

forall j: int {:trigger repeated_str[j * |str| .. (j + 1) * |str|]} | 0 <= j < n
  ensures repeated_str[j * |str| .. (j + 1) * |str|] == str
{
  assert 0 <= j * L;
  assert (j + 1) * L == j * L + L;
  assert j * L <= (j + 1) * L;
  assert j <= n - 1;
  assert j + 1 <= n;
  assert 0 <= n - (j + 1);
  assert n == (j + 1) + (n - (j + 1));
  assert n * L == (j + 1) * L + (n - (j + 1)) * L;
  assert 0 <= (n - (j + 1)) * L;
  assert (j + 1) * L <= n * L;
  assert 0 <= j * L <= (j + 1) * L <= |repeated_str|;
  assert repeated_str[j * L .. (j + 1) * L] == str;
  assert j * |str| == j * L;
  assert (j + 1) * |str| == (j + 1) * L;
  assert repeated_str[j * |str| .. (j + 1) * |str|] == str;
}
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_5_5_repeat/program.dfy(1,7): Error: Verification of 'p_5_5_repeat' timed
  ↳ out after 30 seconds. (the limit can be increased using --verification-time-limit)
|
1 | method p_5_5_repeat(str: string, n: int) returns (repeated_str: string)
|   ~~~~~

```

Dafny program verifier finished with 1 verified, 0 errors, 1 time out

Correct the error in code by altering only the method body. Produce and show only the
 ↳ Dafny body, including the curly braces that surround it. Do not show the signature
 ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnjUuSVNPjxbnG1CeaDp99BsRc3Rc

Model: gpt-5.5-2026-04-23

Created at: 1780745336

Finish reason: length

Usage: 3946 tokens in, and 5462 tokens out

The below is a raw response from GenAI that could not be parsed. It is likely that GenAI got 'stuck' in a loop and hit the pre-set token limit (i.e., finish reason is 'length').

Final Program

```
method p_5_5_repeat(str: string, n: int) returns (repeated_str: string)
  requires n >= 0
  ensures |repeated_str| == n * |str|
  ensures forall i :: 0 <= i < n ==> repeated_str[i * |str| .. (i + 1) * |str| ] ==
    ↪ str
{
  var L := |str|;
  repeated_str := "";
  var k := 0;

  while k < n
    invariant L == |str|
    invariant 0 <= L
    invariant 0 <= k <= n
    invariant |repeated_str| == k * L
    invariant forall j {:trigger repeated_str[j * L .. (j + 1) * L]} :: 0 <= j < k ==> (0 <=
      ↪ j * L <= (j + 1) * L <= |repeated_str| && repeated_str[j * L .. (j + 1) * L]
      ↪ == str)
  {
    var old_repeated := repeated_str;
    assert |old_repeated| == k * L;
    assert forall j {:trigger old_repeated[j * L .. (j + 1) * L]} :: 0 <= j < k ==> (0 <=
      ↪ j * L <= (j + 1) * L <= |old_repeated| && old_repeated[j * L .. (j + 1) * L] ==
      ↪ str);

    repeated_str := old_repeated + str;
    assert repeated_str == old_repeated + str;
    assert |repeated_str| == |old_repeated| + L;
    assert (k + 1) * L == k * L + L;
    assert |repeated_str| == (k + 1) * L;

    forall j: int {:trigger repeated_str[j * L .. (j + 1) * L]} | 0 <= j < k + 1
      ensures 0 <= j * L <= (j + 1) * L <= |repeated_str| && repeated_str[j * L .. (j + 1)
        ↪ * L] == str
    {
      assert j <= k;
      assert 0 <= j * L;
      assert (j + 1) * L == j * L + L;
      assert j * L <= (j + 1) * L;
      assert 0 <= k - j;
      assert k + 1 == (j + 1) + (k - j);
      assert (k + 1) * L == (j + 1) * L + (k - j) * L;
      assert 0 <= (k - j) * L;
      assert (j + 1) * L <= (k + 1) * L;
      assert 0 <= j * L <= (j + 1) * L <= |repeated_str|;

      if j < k {
        var lo := j * L;
        var hi := (j + 1) * L;
        assert j + 1 <= k;
      }
    }
  }
}
```

```

    assert 0 <= k - (j + 1);
    assert k == (j + 1) + (k - (j + 1));
    assert k * L == (j + 1) * L + (k - (j + 1)) * L;
    assert 0 <= (k - (j + 1)) * L;
    assert hi <= k * L;
    assert hi <= |old_repeated|;
    assert old_repeated[lo .. hi] == str;
    assert repeated_str[.. |old_repeated|] == old_repeated;
    assert repeated_str[lo .. hi] == repeated_str[.. |old_repeated|][lo .. hi];
    assert repeated_str[.. |old_repeated|][lo .. hi] == old_repeated[lo .. hi];
    assert repeated_str[lo .. hi] == str;
  } else {
    assert j == k;
    var lo := j * L;
    var hi := (j + 1) * L;
    assert lo == |old_repeated|;
    assert hi == |old_repeated| + L;
    assert hi == |repeated_str|;
    assert repeated_str[|old_repeated| .. |repeated_str|] == str;
    assert repeated_str[lo .. hi] == str;
  }
}

k := k + 1;
}

assert k == n;
assert L == |str|;
assert |repeated_str| == n * |str|;

forall j: int {:trigger repeated_str[j * |str| .. (j + 1) * |str|]} | 0 <= j < n
  ensures repeated_str[j * |str| .. (j + 1) * |str|] == str
{
  assert 0 <= j * L;
  assert (j + 1) * L == j * L + L;
  assert j * L <= (j + 1) * L;
  assert j <= n - 1;
  assert j + 1 <= n;
  assert 0 <= n - (j + 1);
  assert n == (j + 1) + (n - (j + 1));
  assert n * L == (j + 1) * L + (n - (j + 1)) * L;
  assert 0 <= (n - (j + 1)) * L;
  assert (j + 1) * L <= n * L;
  assert 0 <= j * L <= (j + 1) * L <= |repeated_str|;
  assert repeated_str[j * L .. (j + 1) * L] == str;
  assert j * |str| == j * L;
  assert (j + 1) * |str| == (j + 1) * L;
  assert repeated_str[j * |str| .. (j + 1) * |str|] == str;
}
}

```

Total Token Usage

Input tokens: 8120

Output tokens: 25826

Reasoning tokens: 22868

Sum of ‘total tokens’: 33946

Experiment Timings

Overall Experiment started at 1780744715856, ended at 1780745453004, lasting 737148ms (737.15 seconds)

Iteration #4 started at 1780745335275, ended at 1780745453003, lasting 117728ms (117.73 seconds)

Iteration #1 started at 1780744716028, ended at 1780744929549, lasting 213521ms (213.52 seconds)

Iteration #2 started at 1780744929553, ended at 1780745092993, lasting 163440ms (163.44 seconds)

Iteration #3 started at 1780745092993, ended at 1780745335274, lasting 242281ms (242.28 seconds)